08

Part 8 · Streaming Optimization

Incremental processing, state, checkpoints and tuning for steady throughput.

- Structured Streaming
- Watermarks & state
- Checkpointing
- Auto Loader
- Delta writes
- Streaming tuning

8.1 Structured Streaming Fundamentals

★★★

IN ONE LINE

A stream is processed as an ever-growing table in micro-batches, using the same API as batch.

Structured Streaming treats a live data stream as an **unbounded table** and lets you write the same DataFrame / SQL code you would for batch. Spark runs it incrementally as **micro-batches**, each processing the new rows that arrived since the previous batch.

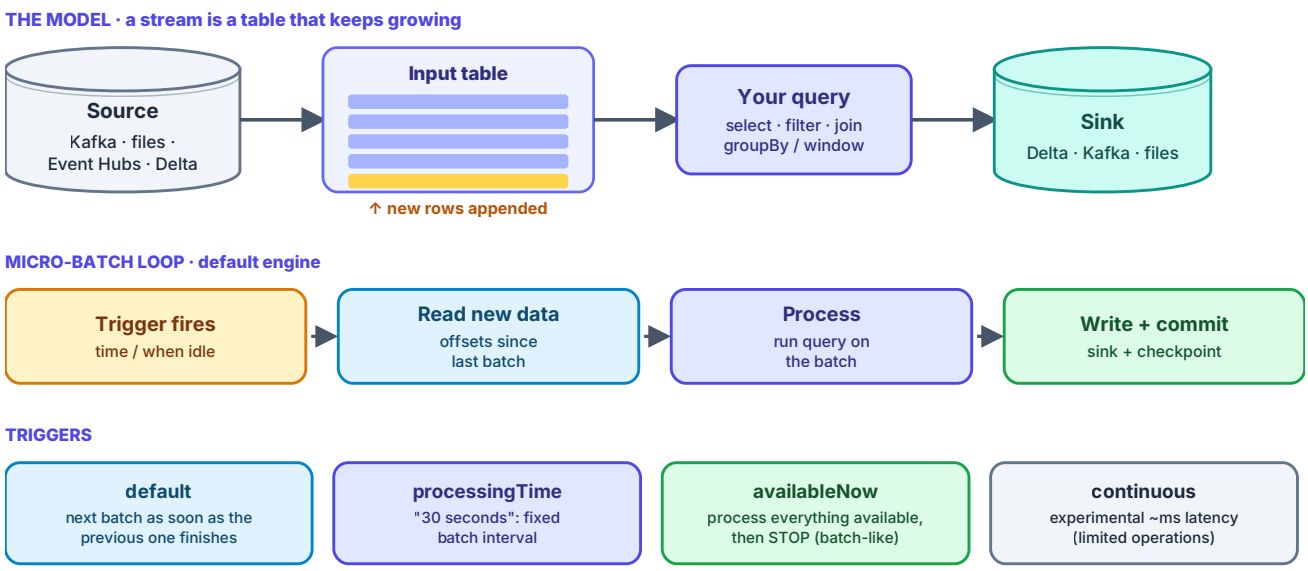


Figure 8.1 — Unbounded table model, micro-batch loop, triggers

A MINIMAL STREAMING QUERY

```
stream = (spark.readStream.format('kafka')
          .option('kafka.bootstrap.servers', 'broker:9092')
          .option('subscribe', 'orders').load())

query = (stream.selectExpr('CAST(value AS STRING) AS json')
         .writeStream.format('delta')
         .option('checkpointLocation', '/chk/orders')
         .trigger(processingTime='30 seconds')
         .start('/lake/bronze/orders'))
```

Concept	Meaning
Latency	Time from event arrival to result; bounded below by the micro-batch interval and batch duration.
Throughput	Rows processed per second; trade against latency (bigger batches are more efficient).
Output mode	append / update / complete: which part of the result is written each batch.
Batch vs stream	Same engine; stream adds a checkpoint, triggers and state for aggregations / joins.

INTERVIEW ANGLE
Say: "Structured Streaming is micro-batch by default, gives **exactly-once** results with a checkpoint and an idempotent / transactional sink (like Delta), and the same code works for batch." Mention **Trigger.AvailableNow** as a cheap incremental-batch pattern.

8.2 Event Time, Watermarks and State

★★★

IN ONE LINE
Watermarks bound how long Spark waits for late data and when it can drop state.

Streams have **event time** (when the event occurred) and **processing time** (when Spark receives it). Events arrive late and out of order. A **watermark** tells Spark how long to wait for late data before finalising a window, which also lets it **discard old state**.

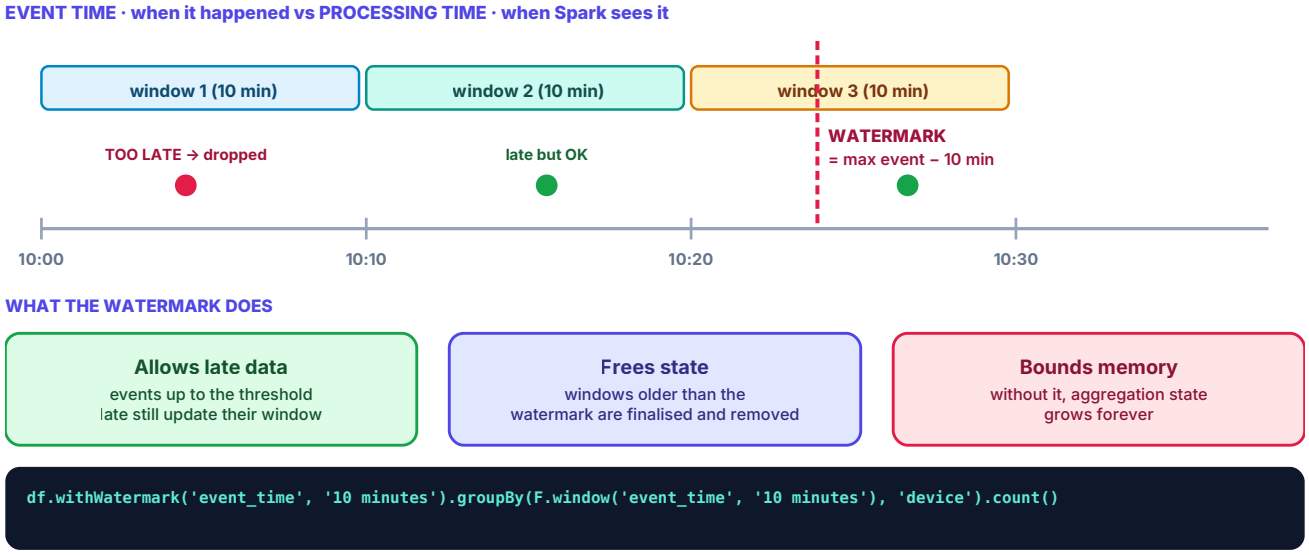


Figure 8.2 — Windows on event time, and the watermark that closes them

Stateful operation	State held	Needs watermark?
Windowed aggregation	Partial result per open window and key	**Yes**, otherwise state grows without limit
Stream-stream join	Buffered rows from each side	**Yes** (with time constraints)
`dropDuplicates`	Seen event ids	Yes, to expire old ids
Stateless (filter, select)	None	No

Choosing the threshold

Larger = more late events accepted but **more state and delay**. Smaller = lower memory and latency but more dropped late data. Base it on observed lateness.

State store

State lives in executor memory and is checkpointed. For large state use the **RocksDB state store** (off-heap, spills to disk) to avoid GC pressure.

INTERVIEW ANGLE
Interview classic: *"Why does my streaming aggregation run out of memory?"* Missing watermark, too long a watermark, or high-cardinality keys: state is unbounded.

8.3 Checkpointing ★★★

IN ONE LINE
Durable record of offsets, commits and state that lets a stream resume exactly where it stopped.

A **checkpoint** is how a streaming query remembers where it is. It stores the **offsets** processed, which batches committed, and any **state**. After a failure or restart, Spark reads the checkpoint and continues exactly where it stopped — this is the basis of **fault tolerance and exactly-once** processing.

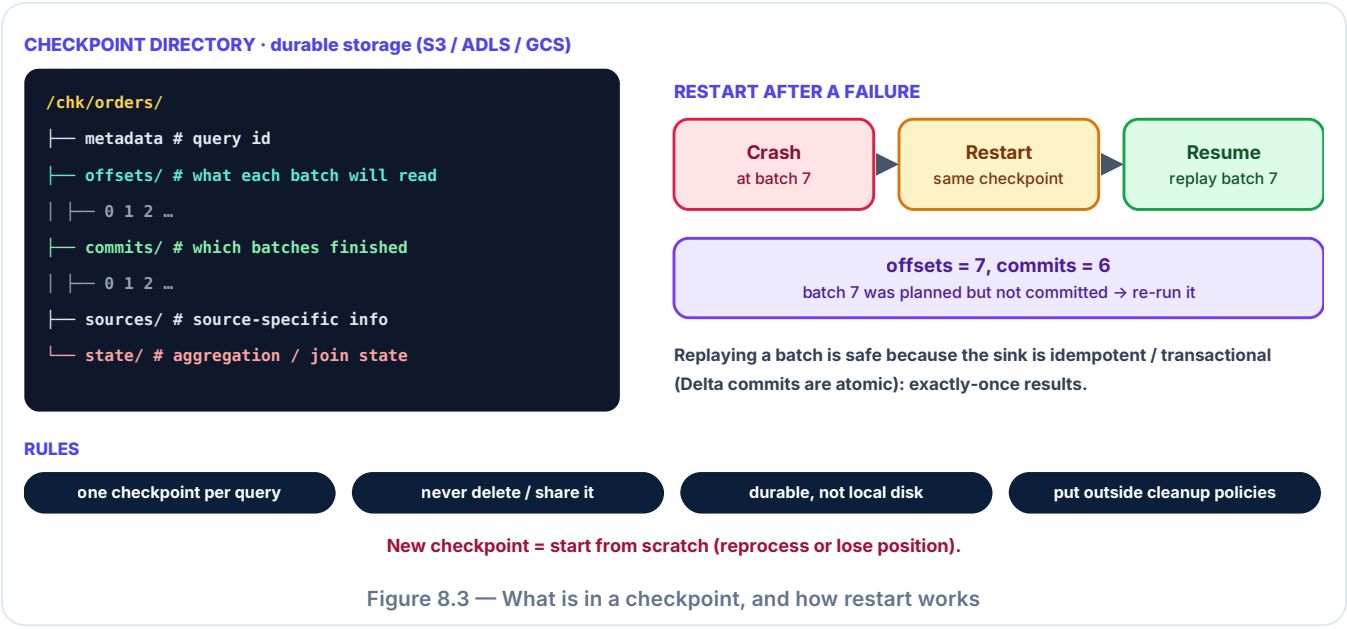


Figure 8.3 — What is in a checkpoint, and how restart works

Do	Do not
Give every query its own checkpoint path	Share a checkpoint between queries
Store it on durable, reliable storage	Use local disk of a cluster that may disappear
Keep it with a lifecycle policy that never deletes it	Let a bucket lifecycle rule clean it (corrupts state)
Plan compatible changes (adding a filter is usually fine)	Change source, stateful logic or shuffle partitions and expect to reuse it

COMMON TRAP
`spark.sql.shuffle.partitions` is **recorded in the checkpoint** for stateful queries. Changing it later does nothing, so size it for future growth before the first run; the only escape is a new checkpoint (reprocess).

INTERVIEW ANGLE
Interview line: *"Checkpointing + replayable source + idempotent sink = exactly-once."* Without a checkpoint a restart would lose or duplicate data.

8.4 Auto Loader

★★★

IN ONE LINE

Incrementally ingests new cloud files with file-state tracking, schema evolution and exactly-once semantics.

Auto Loader (`cloudFiles`) is Databricks' way to ingest **new files from cloud storage incrementally**. It remembers which files it has already processed, so each run reads only what arrived since the last, with **exactly-once** semantics, **schema inference and evolution**, and no manual bookkeeping.

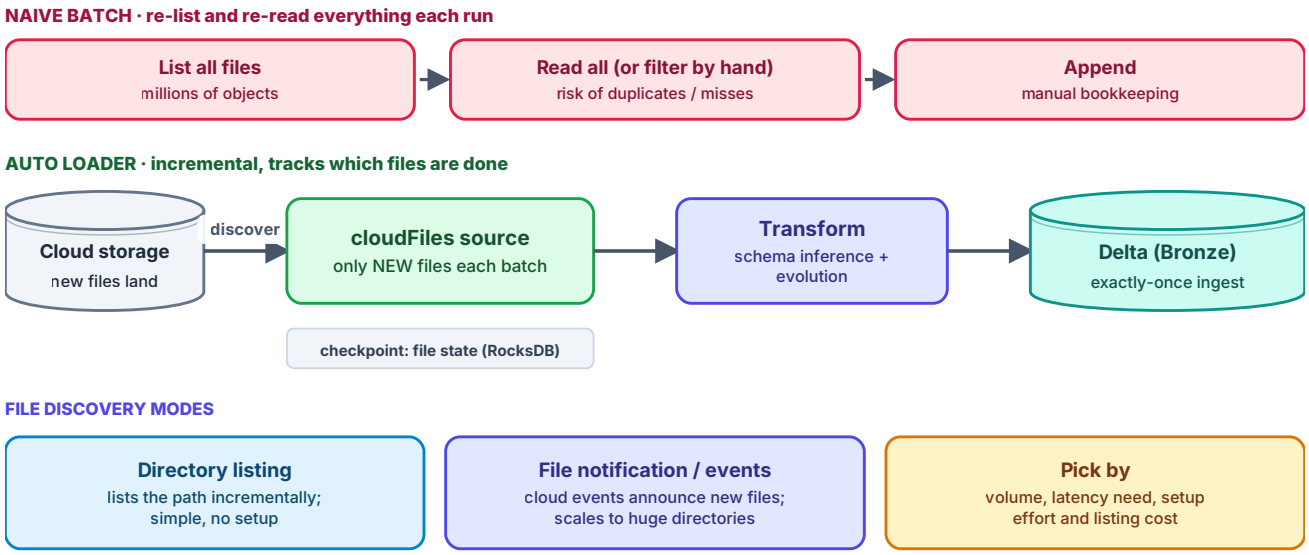


Figure 8.4 — Auto Loader vs naive re-reading, and its discovery modes

AUTO LOADER INGESTION

```
df = (spark.readStream.format('cloudFiles')
      .option('cloudFiles.format', 'json')
      .option('cloudFiles.schemaLocation', '/chk/orders/schema')   # inferred schema + evolution
      .option('cloudFiles.maxFilesPerTrigger', 1000)                # rate limit (default 1000)
      .load('/landing/orders/'))

(df.writeStream.option('checkpointLocation', '/chk/orders/ckpt')
   .trigger(availableNow=True)                # incremental batch; stop when caught up
   .toTable('bronze.orders'))
```

Option	Purpose
`cloudFiles.maxFilesPerTrigger` / `maxBytesPerTrigger`	Rate-limit each micro-batch (files: hard cap, default 1000; bytes: soft cap).
`Trigger.AvailableNow`	Process all files that arrived, then stop: cheap scheduled incremental loads.
Schema evolution + rescued data column	New / mismatched fields are captured instead of failing the pipeline.
`cloudFiles.cleanSource` , `maxFileAge`	Manage processed files and checkpoint state growth (check docs for runtime support).

INTERVIEW ANGLE

For production Databricks recommends running Auto Loader through **Lakeflow pipelines / Jobs** for autoscaling and monitoring. Keep the checkpoint where no lifecycle policy deletes it.

8.5 Streaming Writes to Delta Lake

★★★

IN ONE LINE

Delta gives atomic, exactly-once micro-batch writes; use foreachBatch + MERGE for upserts.

Delta Lake is the standard streaming sink on Databricks: each micro-batch becomes one **atomic transaction**, and together with the checkpoint this yields **exactly-once** writes. Delta tables can also be streaming **sources**, enabling multi-hop Bronze → Silver → Gold streaming.

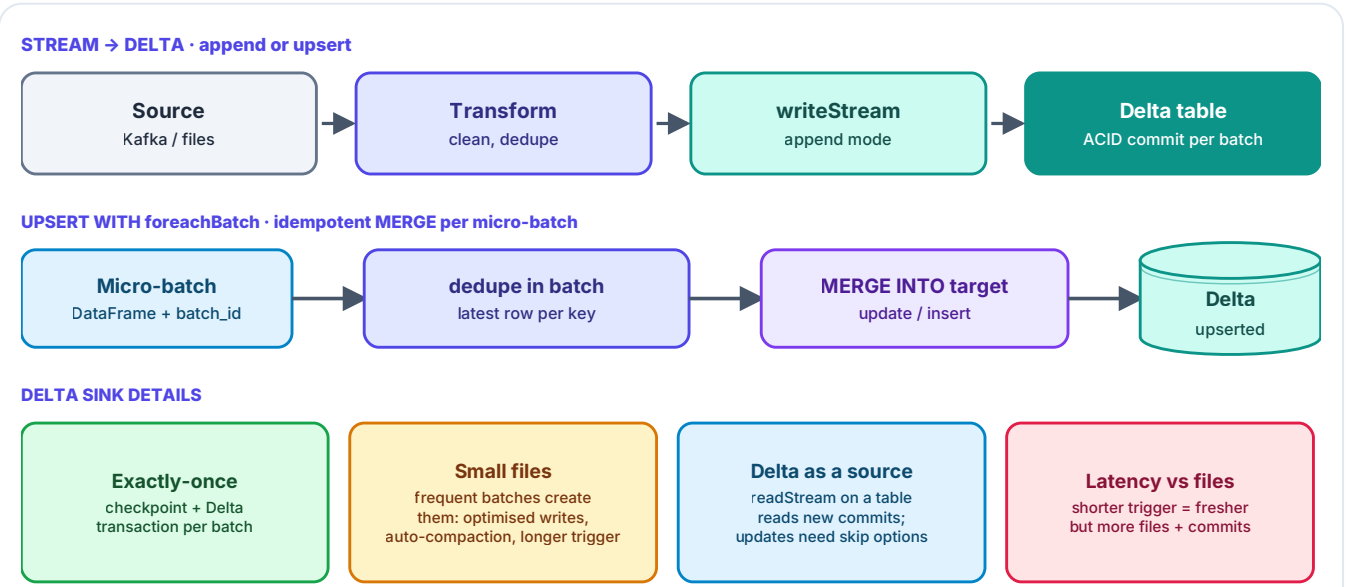


Figure 8.5 — Append and MERGE patterns when streaming into Delta

STREAMING UPSERT WITH FOREACHBATCH

```
def upsert(batch_df, batch_id):
    batch_df.createOrReplaceTempView('updates')
    batch_df.sparkSession.sql('''
      MERGE INTO silver.customers t
      USING (SELECT * FROM updates) s ON t.id = s.id
      WHEN MATCHED THEN UPDATE SET *
      WHEN NOT MATCHED THEN INSERT *''')

(stream.writeStream.foreachBatch(upsert)
       .option('checkpointLocation', '/chk/customers').start())
```

Concern	Guidance
Duplicates within a batch	Deduplicate inside the batch before `MERGE` (a MERGE with duplicate source keys can fail or misapply).
Small files	Enable **optimized writes** and **auto compaction**; use a sensible trigger interval; periodic `OPTIMIZE` .
Replays after failure	`foreachBatch` may re-run a batch: make the write **idempotent** (MERGE or `txnAppId / txnVersion`).
Partitioning	Avoid over-partitioning a stream target: many micro-batches × many partitions = tiny files.

INTERVIEW ANGLE

Streaming into Bronze with **append** and then doing **MERGE** in Silver is the common Medallion pattern (see the Data Engineering handbook).

8.6 Streaming Performance Tuning

★★★

IN ONE LINE

Balancing throughput, latency, state size and cost using per-batch metrics.

Streaming tuning balances **throughput** (rows per second), **latency** (how fresh results are), **state** (memory for windows and joins) and **resources** (cost). Start from the numbers Spark reports for each micro-batch.

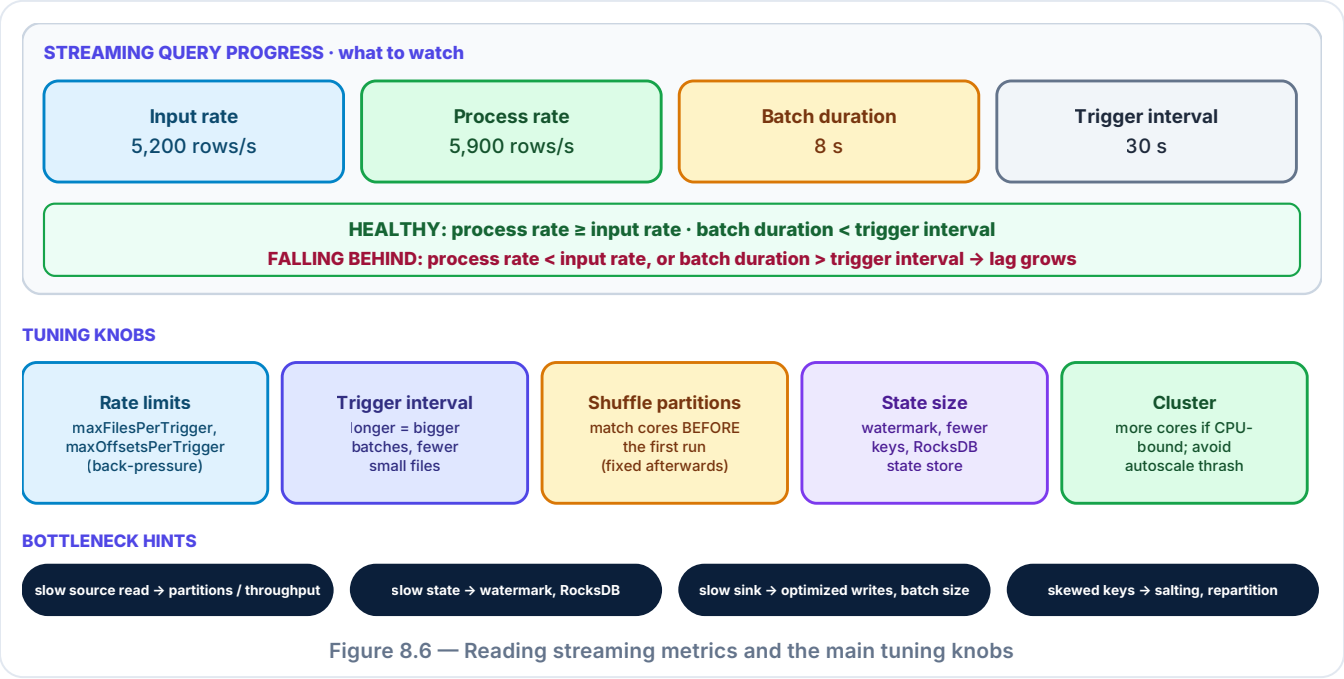


Figure 8.6 — Reading streaming metrics and the main tuning knobs

Problem	Check	Try
Lag keeps growing	Process rate < input rate	Add cores, lower per-batch work, tune partitions, fix skew; rate-limit to stabilise
Too many small files in sink	Files per commit, batch size	Longer trigger, optimized writes / auto compaction
State memory grows	State rows, memory per executor	Add / shorten watermark; RocksDB state store; fewer keys
Batches slow at peaks	Spark UI of the batch job	Increase max cluster size; limit batch size; handle skewed partitions
Need lower latency	Trigger interval, batch duration	Shorter trigger, lighter transforms; consider continuous only if truly needed

INTERVIEW ANGLE

Also remember the **batch alternative**: if minute-level freshness is not required, `Trigger.AvailableNow` on a schedule gives incremental processing at far lower cost than a 24/7 stream.